ROS2 for ATOSFleetManagement Cheat Sheet

1) Source environment
```bash
source /opt/ros/humble/setup.bash
source ~/atos_ws/install/setup.bash
```

2) Build ATOSFleetManagement packages
```bash
cd ~/atos_ws
colcon build --packages-select atos atos_gui --symlink-install
source install/setup.bash
```

3) Start ATOSFleetManagement (normal)
```bash
ros2 launch atos launch_atosfleetmanagement.py insecure:=True
```

4) Start ATOSFleetManagement with 3 simulators
```bash
ros2 launch atos launch_atosfleetmanagement.py insecure:=True
with_truck_simulator:=True
```

Default simulator setup in launch
- `L5S-TRUCK-SIM-1`: `start_index=0`, `target_speed_kmh=80`,
`ignore_warning_speed_commands=True`
- `L5S-TRUCK-SIM-2`: `start_index=250`, `target_speed_kmh=40`
- `L5S-TRUCK-SIM-3`: `start_index=500`, `target_speed_kmh=40`

5) Open GUI
- URL: `http://localhost:8420`
- Go to tab: `RuralRoad Map`
- UI shows live trucks + `Distance To Next Truck Ahead`

6) Run in Docker (same image, with/without simulators)
```bash
cd ~/Documents/repos/ATOS
docker compose -f docker-compose-fleetmanagement.yml up --build
```
With simulators:
```bash
WITH_TRUCK_SIMULATOR=True docker compose -f docker-compose-fleetmanagement.yml up
--build
```
Without simulators:
```bash
WITH_TRUCK_SIMULATOR=False docker compose -f docker-compose-fleetmanagement.yml up
--build
```
Start without rebuild (if image already built):
```bash
docker compose -f docker-compose-fleetmanagement.yml up
```
Start in background:
```bash
docker compose -f docker-compose-fleetmanagement.yml up -d
```

```
When to use `--build`:
- Use `--build` only when image content changed (Dockerfile, dependencies, or source copied into image).
- For normal restart, do **not** use `--build`.

## 7) Run Docker as a systemd service
```bash
sudo mkdir -p /opt/atos
sudo rsync -a ~/Documents/repos/ATOS/ /opt/atos/
sudo cp /opt/atos/scripts/atosfleetmanagement.env.example /etc/default/atosfleetmanagement
sudo cp /opt/atos/scripts/atosfleetmanagement.service /etc/systemd/system/atosfleetmanagement.service
sudo systemctl daemon-reload
sudo systemctl enable --now atosfleetmanagement
```
Service controls:
```bash
sudo systemctl status atosfleetmanagement
sudo systemctl restart atosfleetmanagement
sudo journalctl -u atosfleetmanagement -f
```

8) Copy to another server (what files are needed)
Recommended (safest):
```bash
rsync -a ~/Documents/repos/ATOS/ user@<server>:/opt/atos/
```
Minimum required for Docker-based ATOSFleetManagement:
- `Dockerfile`
- `docker-compose-fleetmanagement.yml`
- `scripts/run_atosfleetmanagement.sh`
- `scripts/installation/`
- `atos/`
- `atos_gui/`
- `atos_interfaces/`
- `conf/`
- Optional for systemd service:
  - `scripts/atosfleetmanagement.service`
  - `scripts/atosfleetmanagement.env.example`

9) Check running nodes
```bash
ros2 node list
```

10) Check key topics
```bash
ros2 topic list | rg "truck_objects|speed_command"
```

11) Watch live truck states for GUI
```bash
ros2 topic echo /atos/truck_objects/state
```

12) Watch control speed commands
```bash

```
ros2 topic echo /atos/truck_objects/speed_command
```

13) TruckObjectControl COT interfaces
- ROS topic input: `/atos/truck_objects/cot` (placeholder format)
- TCP input: `0.0.0.0:8114` listener in TruckObjectControl
- Truck clients connect to: `127.0.0.1:8114` (same machine) or `<host-ip>:8114`
- CoT `<track speed>` is in `m/s` (GUI displays `km/h`)

14) Placeholder ROS COT test (manual)
```bash
ros2 topic pub /atos/truck_objects/cot std_msgs/msg/String "data:
'id=truck_1;distance_m=1000;tcp_connected=1;lat=57.78357;lon=12.76389;speed_mps=2.8
;course_deg=150'" -1
```

15) Start one simulator manually (custom test)
```bash
ros2 run atos atos_truck_simulator --ros-args \
  -p uid:=L5S-TRUCK-UBUNTU \
  -p start_index:=300 \
  -p target_speed_kmh:=35.0 \
  -p acceleration_mps2:=0.6 \
  -p tcp_host:=127.0.0.1 \
  -p tcp_port:=8114
```

16) Simulator option: ignore first-limit slowdown
Use this only on selected trucks:
```bash
-p ignore_warning_speed_commands:=true
```
Behavior:
- Ignores non-zero warning command (for example `30 km/h` at `< 400 m`)
- Still obeys stop command (`0 km/h` at `< 200 m`)

17) Fleet control thresholds
- Gap `< 400 m` => command `target_speed_mps=8.333333`
- Gap `< 200 m` => command `target_speed_mps=0.000000`
- Gap `>= 400 m` => command `target_speed_mps=nochange`

18) TCP speed command manual (per truck)
TruckObjectControl sends one newline-terminated command string per truck over the same TCP connection used by incoming CoT.

Fields:
- `target_speed_mps`: target speed in `m/s`, or `nochange` to keep current truck speed
- `distance_to_truck_ahead_m`: distance in meters from this truck to the next truck ahead in sorted trajectory order
- `truck_ahead_path_index`: current path index for the truck ahead
- `truck_ahead_uid`: UID for the truck ahead
- `reason`: `no_limit`, `min_gap_below_warning_distance`, or `min_gap_below_stop_distance`
- `min_gap_m`: smallest gap found among all connected/fresh trucks in current evaluation cycle
- `connected_count`: number of trucks currently included in control logic

Typical command strings:

```text
target_speed_mps=nochange;distance_to_truck_ahead_m=612.432100;truck_ahead_path_index=310;truck_ahead_uid=L5S-TRUCK-SIM-2;reason=no_limit;min_gap_m=612.432100;connected_count=3
```

```text
target_speed_mps=8.333333;distance_to_truck_ahead_m=154.270000;truck_ahead_path_index=412;truck_ahead_uid=L5S-TRUCK-SIM-3;reason=min_gap_below_warning_distance;min_gap_m=154.270000;connected_count=3
```

```text
target_speed_mps=0.000000;distance_to_truck_ahead_m=92.140000;truck_ahead_path_index=418;truck_ahead_uid=L5S-TRUCK-SIM-1;reason=min_gap_below_stop_distance;min_gap_m=92.140000;connected_count=3
```

19) Stop everything
- Press `Ctrl+C` in launch terminal